

SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE



Objectives

01

Inheritance

02

Runtime
Polymorphism

03

Abstract Class



✓ Formally Defining Inheritance

- ✓ ● Inheritance is a core OOP concept that allows a class to acquire the accessible properties and behaviors from another class
- ✓ ● It helps in creating a new class from an existing class
- ✓ ● Promotes code reusability and better organization



✓ Key Terminologies

- ✓ ● The newly created class is known as subclass/child/derived class
- ✓ ● The existing class from where the child class is derived is known as superclass/parent/base class
- ✓ ● The `extends` keyword is used to perform inheritance in Java

Key Terminologies

- ✓ • Child class can use the super keyword to call constructors or methods of the parent class
- ✓ • Inheritance models an Is-A relationship between classes



✓ Types of Inheritance

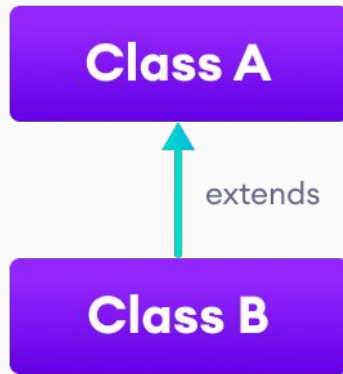
There are four main types of inheritance.

- ✓ 1. Single Inheritance
- ✓ 2. Multilevel Inheritance
- ✓ 3. Hierarchical Inheritance
- ✓ 4. Multiple Inheritance



✓ 1. Single Inheritance

In single inheritance, a class inherits from one parent class only

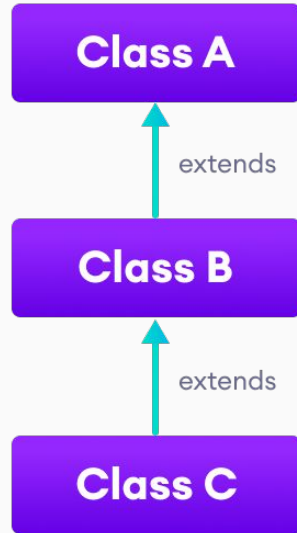


```
class Parent {  
    // fields and methods  
}  
  
class Child extends Parent {  
    // additional fields and methods  
}
```



✓ 2. Multilevel Inheritance

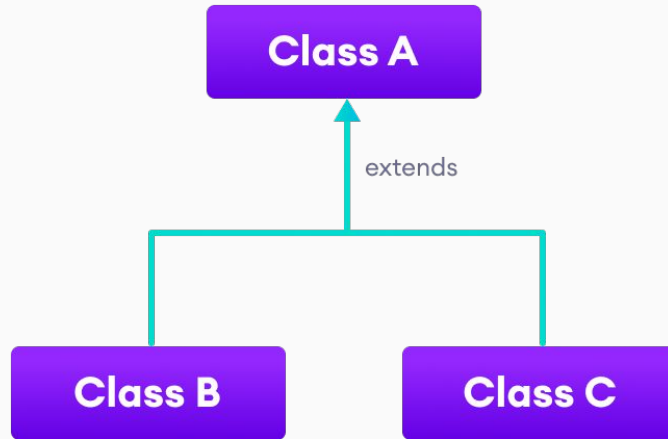
A subclass extends from a superclass and then the same subclass acts as a superclass for another class



```
class Grandparent {  
    // Grandparent class code  
}  
  
class Parent extends Grandparent {  
    // Parent class code  
}  
  
class Child extends Parent {  
    // Child class code  
}
```

3. Hierarchical Inheritance

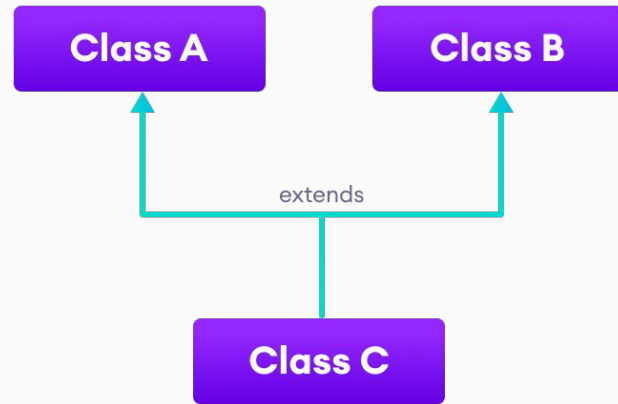
In hierarchical inheritance, multiple classes inherit from a single parent class



```
class Parent {  
    // Parent class code  
}  
  
class Child1 extends Parent {  
    // Child1 class code  
}  
  
class Child2 extends Parent {  
    // Child2 class code  
}
```

4. Multiple Inheritance

In multiple inheritance, a single subclass extends from multiple superclasses



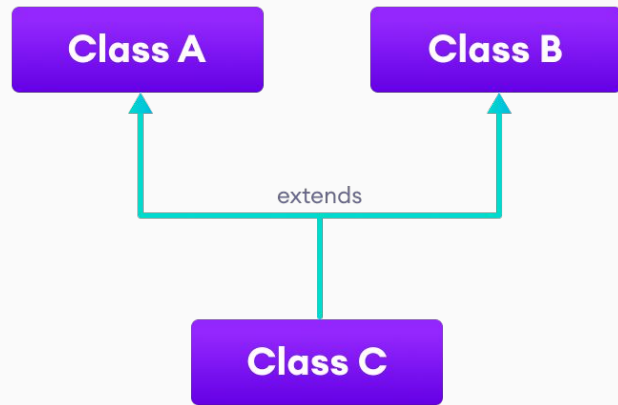
Java does not support multiple inheritance.

However, this can be achieved using interfaces.

An interface is a blueprint that defines a set of methods a class must implement without providing full implementation details.

4. Multiple Inheritance

In multiple inheritance, a single subclass extends from multiple superclasses



```
interface A {  
    void methodA();  
}  
  
interface B {  
    void methodB();  
}  
  
class C implements A, B {  
    public void methodA() {  
        System.out.println("Method A");  
    }  
  
    public void methodB() {  
        System.out.println("Method B");  
    }  
}
```

✓ Benefits of Inheritance

* Code Reusability

Promotes the reuse of existing code, reducing redundancy and improving efficiency

Improved Maintenance

By centralizing common features in a parent class, inheritance makes it easier to manage and update the code

Enables Polymorphism

Inheritance helps us achieve runtime polymorphism through method overriding and subtyping

Polymorphism Through Inheritance

Inheritance powers polymorphism through three interconnected mechanisms:

1. **Upcasting:** A superclass reference can refer to a subclass object
2. **Method Overriding:** A subclass provides its own implementation of an inherited method
3. **Dynamic Binding:** The JVM chooses the overridden method to execute based on the actual object type at runtime



Upcasting

- ✓ • Upcasting is the process of casting a subclass object to a superclass reference type
- ✓ • It is also known as "widening"
- ✓ • A child class inherits everything from its parent, a child object is-a parent object, making this process completely safe and automatic

Upcasting

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

```
class Dog extends Animal {  
    String breed = "Labrador";  
  
    void bark() {  
        System.out.println("Woof!");  
    }  
}
```

```
Animal a = new Dog();
```

This is valid.

We can treat an object of the child class as if it belongs to the parent class.

Because Dog has all the characteristics of Animal.

The Animal reference type can hold different subclass objects and exhibit different behaviors through polymorphism



✓ Method Overriding

- ✓ ● Allows a subclass to provide a specific implementation of a method that is already defined in its parent class
- ✓ ● Achieved when child and parent classes have methods with the same signature
- ✓ ● JVM determines which method will be called based on the **object** type



Method Overriding

```
class Animal {  
    public void displayInfo() {  
        System.out.println("I am an animal.");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    public void displayInfo() {  
        System.out.println("I am a dog.");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Dog d1 = new Dog();  
        d1.displayInfo(); // I am a dog  
    }  
}
```

displayInfo() method is present in both the Animal superclass and the Dog subclass.

When we call displayInfo() using the d1 object (object of the subclass), the method inside the subclass Dog is called.



✓ Method Overriding Rules

- ✓ • Name and parameter list must match the parent method
- ✓ • Access modifier cannot be more restrictive than the parent method
- ✓ • Static and final methods cannot be overridden
- ✓ • The @Override annotation catches mistakes like typos in method names



Special Cases

```
class Animal {  
    public void displayInfo() {  
        System.out.println("I am an animal.");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    public void displayInfo() {  
        super.displayInfo();  
        System.out.println("I am a dog.");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Dog d1 = new Dog();  
        d1.displayInfo();  
  
        // I am an animal  
        // I am a dog  
    }  
}
```

super can be used to call the parent method



Special Cases

```
class Parent{  
  
    // Can't be overridden  
    public final void show(){  
  
    }  
}
```

If we don't want a method to be overridden, we declare it as final

```
class Child extends Parent{  
  
    // This would produce error  
    public void show() {}  
}
```



Special Cases

```
class Parent{
    private void display(){
        System.out.println("Private method");
    }
}

class Child extends Parent{
    void display(){
        // This is a new method
        System.out.println("Child method");
    }
}

public class Main{

    public static void main(String[] args){

        Child c = new Child();

        // Calls Child's method
        c.display();
    }
}
```

Private methods cannot be overridden because they are not visible to subclasses. A subclass method with the same name is treated as a new, independent method, unrelated to the parent class.



Special Cases

```
class Animal {  
    static void type() {  
        System.out.println("I am an Animal");  
    }  
}
```

```
class Dog extends Animal {  
    static void type() {  
        System.out.println("I am a Dog");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        Animal a = new Dog();  
        a.type(); // "I am an Animal"  
  
        Dog d = new Dog();  
        d.type(); // "I am a Dog"  
    }  
}
```

Static methods cannot be overridden.

Static method calls are resolved at compile time.

✓ Abstract Class

- ✓ An abstract class in Java is a class that cannot be instantiated on its own
- ✓ It is designed to serve as a blueprint for other classes
- ✓ Abstract classes are declared using the abstract keyword

Abstract class: It is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

Abstract method: It can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).



Abstract Class

It may contain:

- ✓ ● Abstract methods (methods without a body)
- ✓ ● Concrete methods (methods with implementation)
- ✓ ● Constructors
- ✓ ● Instance variables
- ✓ ● Static and final methods



Abstract Class

```
abstract class Animal {  
    abstract void makeSound(); // no body – abstract method  
    void breathe() {  
        System.out.println("Breathing..."); // regular, concrete method  
    }  
}
```



Features of Abstract Classes

Cannot be instantiated

```
abstract class Animal { }

public class Main {
    public static void main(String[] args) {
        Animal a = new Animal(); // Error
    }
}
```



Features of Abstract Classes

Even though they cannot be instantiated, they can have constructors

```
abstract class Animal {  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
}
```



Features of Abstract Classes

An abstract class may exist without an abstract method

```
abstract class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```



Features of Abstract Classes

Even if it has one abstract method, the class must be declared
abstract

```
class Animal { // compilation error  
    abstract void makeSound();  
}
```

```
abstract class Animal { // correct  
    abstract void makeSound();  
}
```



Features of Abstract Classes

- ✓ • Support Inheritance
- ✓ • Abstract classes can extend other classes and be extended by other classes
- ✓ • A subclass must override all abstract methods of an abstract class
- ✓ • However, if the subclass is declared abstract, it's not mandatory to override abstract methods



Features of Abstract Classes

```
abstract class Animal {  
    abstract void makeSound();  
}
```

```
class Dog extends Animal {  
  
    @Override  
    void makeSound() {  
        System.out.println("Woof!"); // override the abstract method  
    }  
}
```

OR

```
abstract class Dog extends Animal { } // make Dog abstract
```



Features of Abstract Classes

- ✓ Can have static and final methods
- ✓ Static methods belong to the class
- ✓ Final methods are inheritable but not customizable (cannot be overridden)



Features of Abstract Classes

```
abstract class Animal {  
  
    static void info() {  
        System.out.println("Animal  
        information");  
    }  
  
    final void sleep() {  
        System.out.println("Sleeping");  
    }  
}  
  
class Dog extends Animal {  
  
    // @Override // Error! Cannot override  
    // void sleep() { }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Animal.info(); // Animal information  
  
        Dog d = new Dog();  
  
        d.sleep();      // Sleeping  
    }  
}
```



Features of Abstract Classes

Although they cannot be instantiated, they can hold subclass objects

```
abstract class Animal {  
  
    abstract void makeSound();  
  
}  
  
class Dog extends Animal {  
  
    @Override  
    void makeSound() {  
        System.out.println("Woof!");  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Animal a = new Dog();  
  
        a.makeSound(); // Output: Woof!  
    }  
}
```

✓ When to use Abstract Classes

Just like when to use
Inheritance

- ✓ ● Multiple related classes share common attributes and behaviors
- ✓ ● Some methods should be common to all subclasses, while others require different implementations
- ✓ ● You want to prevent direct instantiation of a general class
- ✓ ● You want to provide a common parent type for polymorphism



Scenario: Ride Sharing App

A ride-sharing company offers BikeRide and CarRide services. Every ride has a passenger, pickup location, a destination, and can be started and completed similarly. However, fare calculation and other characteristics may differ between ride types.

Design an object-oriented solution using an abstract class and appropriate subclasses. Identify the common attributes and methods, and the methods that should be implemented differently by each ride type.